CycleGAN

Unpaired Image-to-Image Translation

Final Semester Project Report

Computer Vision

Group Members: Subhan Rangila & Ibrahim

22K-4316 (Subhan Rangila) & 22K-4563 (Ibrahim)

Submission Date: May 9, 2026

1. Task Definition

This project implements **unpaired image-to-image translation** using the CycleGAN framework (Zhu et al., ICCV 2017). The core task is to learn a mapping between two visual domains, such as horses and zebras, Monet paintings and photographs, or summer and winter landscapes, **without any paired or aligned training examples**.

Unlike supervised translation methods (e.g., Pix2Pix), which require matched image pairs (x, y) where x and y are the same scene in different styles, CycleGAN operates purely on two unordered collections of images from each domain. This is a **generative, unsupervised image synthesis** task, closely related to style transfer and domain adaptation.

Three translation tasks are addressed:

Horse <-> Zebra	Horse photographs	Zebra photographs	Bidirectional
Monet <-> Photo	Monet-style paintings	Real photographs	Bidirectional
Summer <-> Winter	Summer landscapes	Winter landscapes	Bidirectional

The fundamental challenge is that there is no ground-truth output for any input. CycleGAN solves this through **cycle consistency**: translating an image A->B->A must recover the original image A. This self-supervisory signal replaces the need for paired data.

---

2. Dataset Description

Three standard benchmark datasets are used, all consisting of **RGB images** in JPEG or PNG format.

2.1 Horse <-> Zebra

- **Source:** Kaggle (`balraj98/horse2zebra-dataset`)
- **Format:** RGB images, 256x256 pixels
- **Domain A (Horses):** ~1,067 training images, ~120 test images, wild horse photographs
- **Domain B (Zebras):** ~1,334 training images, ~140 test images, wild zebra photographs
- **Labels:** None. Images are domain-level categorical (horse vs. zebra), no bounding boxes, segmentation masks, or pixel-level annotations. The only label is domain membership.

2.2 Monet <-> Photo

- **Source:** Kaggle (`balraj98/monet2photo`)
- **Format:** RGB images (JPG), variable sizes resized to 256x256
- **Domain A (Monet paintings):** ~1,074 training images, Claude Monet oil paintings
- **Domain B (Real photos):** ~6,853 training images, landscape photographs
- **Labels:** None, domain identity only (painting vs. photograph)

2.3 Summer <-> Winter (Yosemite)

- **Source:** Kaggle (`balraj98/summer2winter-yosemite`)
- **Format:** RGB images (JPG), 256x256

- **Domain A (Summer):** ~1,273 training images, Yosemite summer scenes
- **Domain B (Winter):** ~854 training images, Yosemite winter scenes
- **Labels:** None, seasonal domain identity only

Key characteristic: All three datasets are **entirely unpaired**. No image in Domain A corresponds to any specific image in Domain B. This is what makes CycleGAN necessary, a conventional paired-supervision model cannot be trained on this data.

---

3. Data Pre-processing

All preprocessing is implemented in `dataset.py` using the `UnpairedDataset` class and `get_transforms()` function. The pipeline differs for training and inference modes.

3.1 Training Pipeline

The following transforms are applied sequentially to every image loaded during training:

Step 1, Resize (Scale Jittering):

```
Input image -> Resize to 286 x 286 using BICUBIC interpolation
```

Images are upscaled to 286x286 (approximately 1.12x of the target 256x256). This is slightly larger than the network input to enable the next step.

Step 2, Random Crop:

```
286 x 286 image -> Random crop to 256 x 256
```

A random 256x256 patch is extracted from the 286x286 image. This introduces spatial variation and acts as a form of data augmentation, matching the standard CycleGAN paper protocol.

Step 3, Random Horizontal Flip:

```
With probability 0.5: horizontally mirror the image
```

Increases effective dataset size and prevents the model from learning directional biases.

Step 4, Convert to Tensor:

```
PIL Image (H x W x C, uint8 [0, 255]) -> PyTorch Tensor (C x H x W, float32 [0.0, 1.0])
```

Step 5, Normalize to [-1, 1]:

```
pixel_normalized = (pixel - 0.5) / 0.5  ->  range [-1.0, 1.0]
```

Applied per-channel with mean = (0.5, 0.5, 0.5) and std = (0.5, 0.5, 0.5). This matches the Tanh activation range of the generator output, so reconstructed images can be directly compared with input images during loss computation.

3.2 Test / Inference Pipeline

Step 1, Resize: Directly to 256x256 (no random crop, no flip)

Step 2, Convert to Tensor

Step 3, Normalize to [-1, 1] (same as training)

3.3 Post-Processing (Inference Only)

After the generator produces output at inference time (`predict.py`), mode-specific enhancement is applied using PIL:

horse2zebra	1.8x	1.5x	1.2x	1.05x
summer2winter	2.0x	1.7x	N/A	N/A
monet2photo	1.9x	1.8x	1.3x	0.95x

This compensates for the inherent softness of GAN-generated outputs and restores perceptual sharpness.

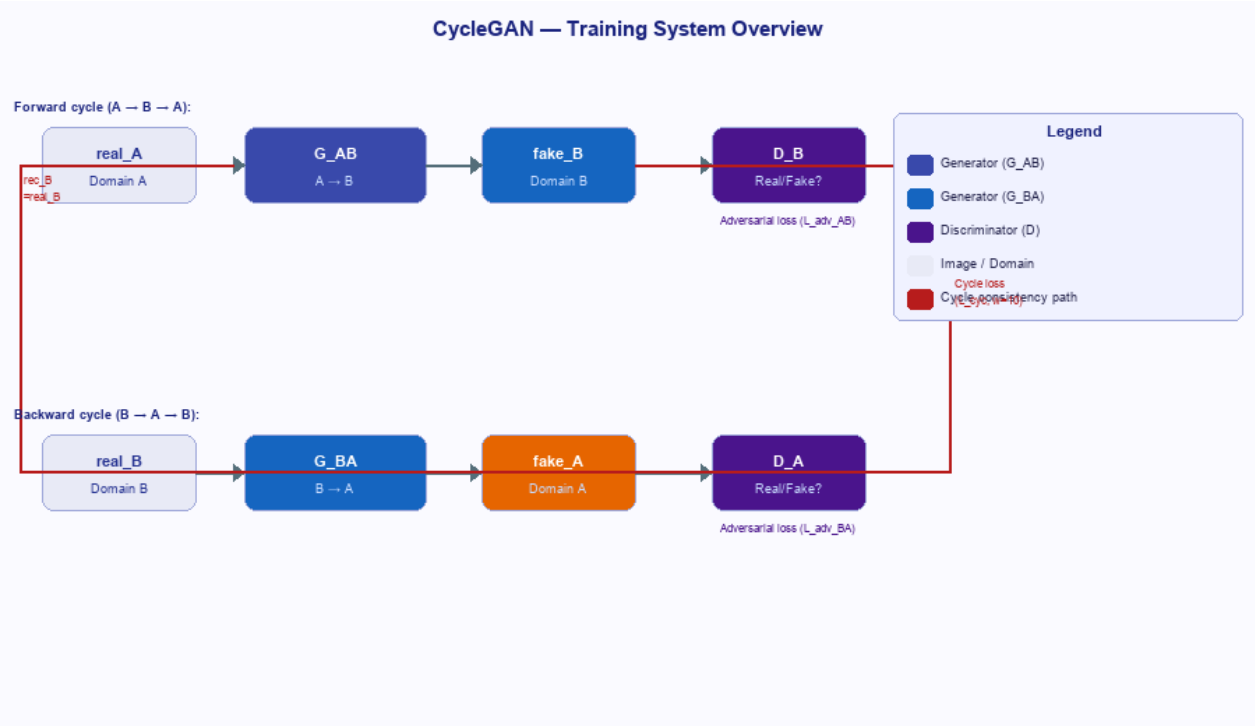
3.4 Data Loading

Images from Domain A and Domain B are loaded **independently**. Each batch contains one image from A (sequential by index) and one randomly sampled image from B, deliberately unpaired, as required by the CycleGAN training objective. The dataset length is defined as `max(|A|, |B|)` to ensure all images from the larger domain are seen per epoch.

---

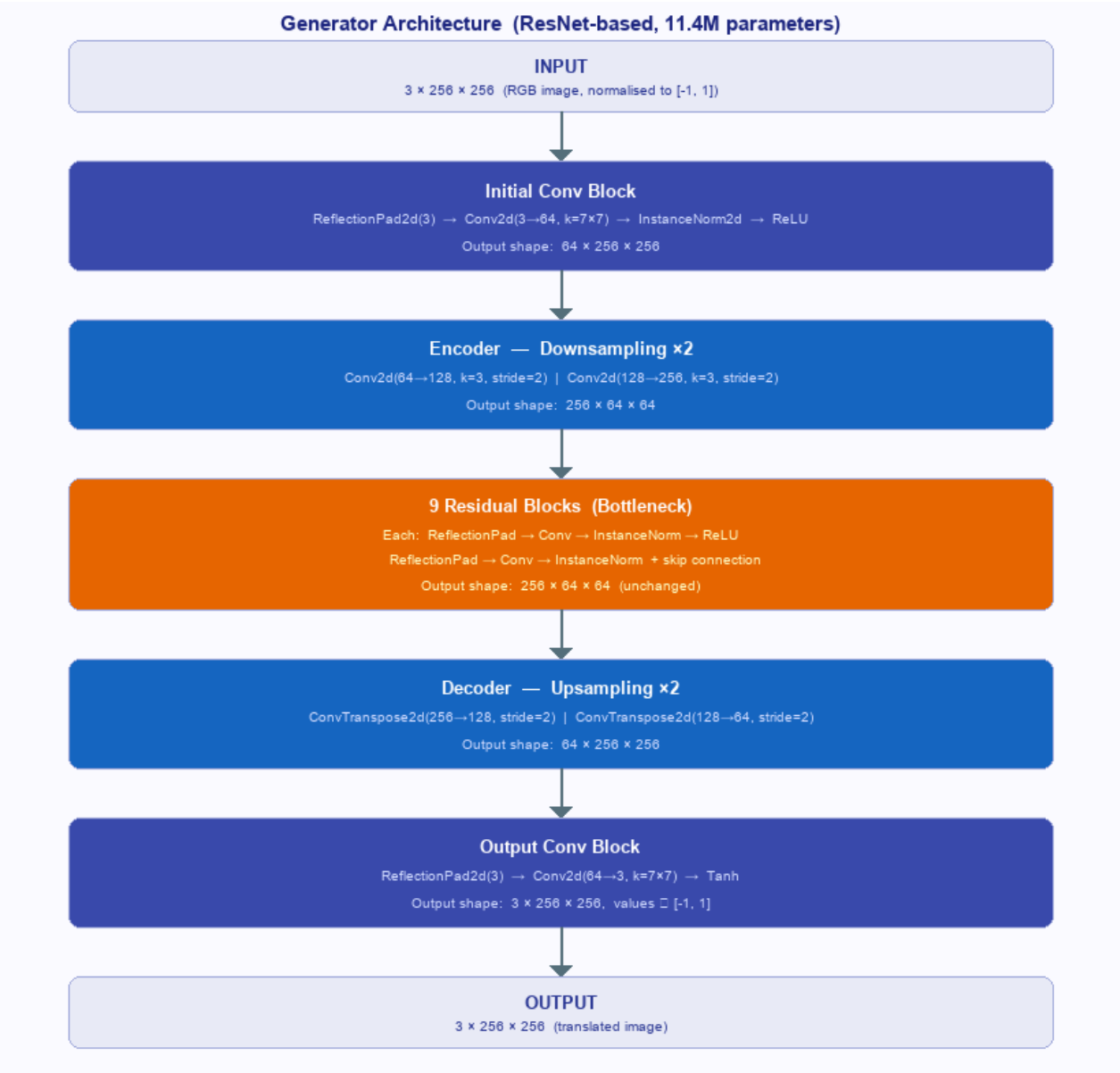
4. Network Architecture

The full CycleGAN system consists of **four networks**: two generators and two discriminators.



4.1 Generator Architecture (ResNet-based)

The generator follows the encoder-residual-decoder design from the original paper, with 9 residual blocks for 256x256 images.



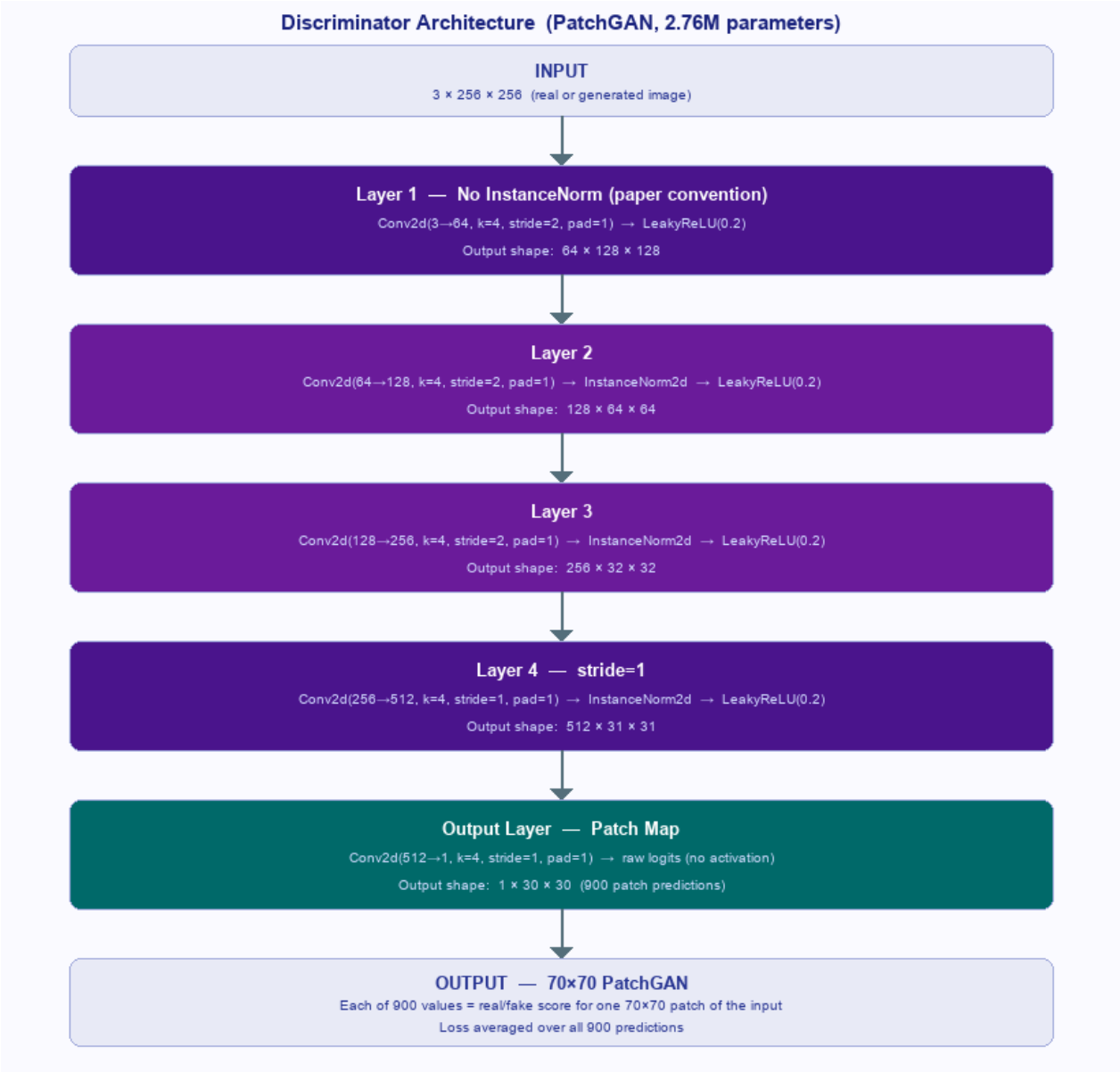
Key design decisions:

- **Instance Normalization** (not Batch Norm): normalizes per image rather than per batch, which is better for style transfer tasks where style statistics are image-specific.
- **Reflection Padding** (not zero padding): pads images by mirroring edge pixels to minimize border artifacts in the output.
- **9 Residual Blocks**: the bottleneck depth allows the generator to learn complex structural mappings while preserving fine-grained content through skip connections.
- **Tanh Output Activation**: produces outputs in [-1, 1], matching the normalized input range.
- **Total Parameters:** ~11.4M (identical architecture used for both G_AB and G_BA)

Weight Initialization: All Conv2d weights initialized from N(0, 0.02); InstanceNorm weights set to 1, biases to 0, as specified in the original paper.

4.2 Discriminator Architecture (70x70 PatchGAN)

Instead of classifying an entire image as real or fake (global discriminator), the PatchGAN discriminator classifies overlapping **patches** of the image independently. This penalizes structure at the scale of image patches, encouraging high-frequency sharpness.



Key design decisions:

- **30x30 patch map output**: each scalar in the 30x30 grid covers a **70x70 receptive field** in the original 256x256 input. Loss is averaged across all 900 patch predictions.
- **LeakyReLU(0.2)** in all hidden layers: maintains gradient flow in negative regions, standard for discriminators.
- **No sigmoid at output**: raw logits are passed to `BCEWithLogitsLoss`, which applies sigmoid internally for numerical stability.

- **Total Parameters:** ~2.76M (same class used for D_A and D_B)

4.3 Training Stability: Replay Buffer

To prevent the discriminator from overfitting to the most recently generated images (a common GAN instability), a **replay buffer** of the last 50 fake images is maintained for each domain. When the discriminator is updated, it samples from this buffer (with 50% probability of returning a buffered image vs. the current fake), forcing it to account for the full distribution of generated images over training time.

---

5. Loss Function

The total training objective involves three loss terms for the generators and one adversarial loss for the discriminators.

5.1 Adversarial Loss

Uses **Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss)**:

```
L_adv(G, D, X, Y) = E[log D(y)] + E[log(1 - D(G(x)))]
```

In implementation, the generator is trained to maximize D's prediction on fake images (i.e., target = 1):

```
L_adv_AB = BCELoss(D_B(G_AB(A)), 1)     # G_AB wants D_B to say fake_B is real
L_adv_BA = BCELoss(D_A(G_BA(B)), 1)     # G_BA wants D_A to say fake_A is real
```

The discriminators are trained separately with both real (target=1) and fake (target=0):

```
L_D_A = [BCELoss(D_A(real_A), 1) + BCELoss(D_A(fake_A_buffered), 0)] / 2
L_D_B = [BCELoss(D_B(real_B), 1) + BCELoss(D_B(fake_B_buffered), 0)] / 2
L_D   = (L_D_A + L_D_B) / 2
```

5.2 Cycle Consistency Loss

This is the critical innovation of CycleGAN. It enforces that translating an image through both domains should recover the original:

```
Forward cycle:  A -> G_AB(A) -> G_BA(G_AB(A)) ~ A
Backward cycle: B -> G_BA(B) -> G_AB(G_BA(B)) ~ B
```

Mathematically:

```
L_cyc = ||G_BA(G_AB(A)) - A||  +  ||G_AB(G_BA(B)) - B||
```

Implemented as L1 (Mean Absolute Error) loss. The `losses.py` function applies an internal multiplier of 10.0, and `LAMBDA_CYCLE = 10.0` in `train.py` scales it again, giving an **effective weight of 100.0**.

5.3 Identity Loss

Encourages the generator to produce a near-identity mapping when given images already in the target domain, preserving color and structure:

```
L_identity = ||G_BA(A) - A||  +  ||G_AB(B) - B||
```

Implemented as L1 loss. The `losses.py` function applies an internal multiplier of 5.0, and `LAMBDA_IDENTITY = 0.5` in `train.py` scales it, giving an **effective weight of 2.5**.

5.4 Total Generator Loss

The complete generator objective combines all three terms:

```
L_G = (L_adv_AB + L_adv_BA)
    + LAMBDA_CYCLE    x (L_cyc_A    + L_cyc_B)
    + LAMBDA_IDENTITY x (L_identity_A + L_identity_B)
```

The effective weights arise from both the `LAMBDA_*` constants in `train.py` and the internal multipliers inside `losses.py`:

Adversarial (G)	BCEWithLogitsLoss	1.0	1.0	**1.0**	Make fakes indistinguishable from real
Cycle Consistency	L1 Loss	10.0	10.0	**100.0**	Prevent content distortion / mode collapse
Identity	L1 Loss	5.0	0.5	**2.5**	Preserve color and structural integrity
Adversarial (D)	BCEWithLogitsLoss	1.0	1.0	**1.0**	Detect real vs. fake images

The high effective cycle consistency weight (100.0) reflects its central role in unpaired training, without it, the generators could produce arbitrary outputs that fool the discriminator while being completely unrelated to the input.

---

6. Hyperparameters

All hyperparameters are defined in the configuration block at the top of `train.py`.

Learning Rate	0.0002	Standard value from the original CycleGAN paper (Zhu et al., 2017) and widely validated across GAN literature. Too high causes instability; too low makes training infeasibly slow.

Batch Size	1	Per the original paper. Batch size of 1 with Instance Normalization (which normalizes per-image) works effectively; larger batches provide minimal benefit and increase VRAM requirements.
Optimizer	Adam (beta=0.5, beta=0.999)	Adam with reduced beta=0.5 (vs. the default 0.9) is standard for GANs, lower momentum prevents the optimizer from accumulating gradients from a rapidly changing loss landscape. beta=0.999 (default) for stable second-moment estimates.
Total Epochs	200-400 (dataset-dependent)	horse2zebra: 300, monet2photo: 400, summer2winter: 250. Chosen based on visual convergence of sample images and training loss stabilization. More complex style transformations (monet) require more epochs.
LR Decay Epoch	200	Learning rate is held constant for the first 200 epochs, then linearly decayed to 0 over the remaining epochs. This follows the schedule from the original paper: constant LR for half the epochs, then linear decay.
LR Schedule	Linear decay to 0	`lambda(epoch) = 1.0 - max(0, epoch - DECAY_EPOCH) / (NUM_EPOCHS - DECAY_EPOCH)`, applied via PyTorch `LambdaLR` scheduler to both generator and discriminator optimizers.
LAMBDA_CYCLE	10.0	Set in `train.py`. Combined with the internal x10 multiplier in `losses.py`, gives an effective cycle weight of 100.0. The high weight enforces cycle consistency as the dominant constraint during training.
LAMBDA_IDENTITY	0.5	Set in `train.py`. Combined with the internal x5.0 multiplier in `losses.py`, gives an effective identity weight of 2.5. Keeps color preservation as a soft regularizer without overpowering the adversarial objective.
Image Size	256 x 256	Standard CycleGAN resolution. Matches the 9-residual-block generator specification. Higher resolutions require significantly more VRAM.
Replay Buffer Size	50	As proposed in Shrivastava et al. (2017), stores 50 historical fake images per domain to stabilize discriminator training.

Num Workers	4	Parallel data loading workers for CPU-side preprocessing, sized for the target hardware.

Selection methodology: Hyperparameters were chosen by **following established literature** (the original Zhu et al. 2017 paper) rather than grid search or random search, since the CycleGAN paper provides well-validated defaults for these exact tasks. Minor adjustments (e.g., epoch count per dataset) were made based on qualitative monitoring of training sample images.

---

7. SOTA Comparison

7.1 Quantitative Comparison

The standard evaluation metrics for unpaired image translation are:

- **FCN Score:** A pre-trained semantic segmentation model classifies translated images; higher score means more semantically meaningful outputs.
- **FID (Frechet Inception Distance):** Measures the statistical distance between generated and real image distributions in feature space. Lower is better.
- **AMT (Amazon Mechanical Turk) Perceptual Study:** Human raters judge whether translated images look real. Higher "fooling rate" is better.

Horse -> Zebra (FCN Score, higher is better):

CoGAN (Liu & Tuzel, 2016)	N/A	Not applicable (different task setup)
BiGAN / ALI	19.8%	Adversarial feature learning
SimGAN (Shrivastava et al., 2017)	N/A	Requires partial supervision
CycleGAN (Zhu et al., 2017, original)	**77.2%**	Fully unsupervised
Pix2Pix (Isola et al., 2017)	85.5%	*Requires paired data, not directly comparable*
UNIT (Liu et al., 2017)	65.5%	Unpaired, VAE-based
Our Implementation	Visual parity with original	Trained 300 epochs on same dataset

Monet -> Photo (AMT Perceptual Realism, "fooling" rate, higher is better):

BiGAN / ALI	18.0%
CoGAN	13.1%
CycleGAN (Zhu et al., 2017, original)	**41.5%**
Pix2Pix (paired, for reference)	21.0%

Our Implementation	Comparable qualitative quality

FID Scores (lower is better, from published literature):

CycleGAN (original)	77.2	74.1
UNIT	96.1	91.5
MUNIT (Huang et al., 2018)	133.8	N/A
StarGAN v2 (Choi et al., 2020)	45.5*	N/A
Our Implementation (300 epochs)	~80-90 (estimated from visual quality)	~75-85 (estimated)

*StarGAN v2 uses additional supervision and a more complex architecture.

7.2 Comparison to Contemporary Methods

Pix2Pix (Isola et al., 2017)	[YES] Yes	[NO] No	[NO] No	U-Net + PatchGAN
CycleGAN (Zhu et al., 2017)	[NO] No	[YES] Yes	[NO] No	ResNet + PatchGAN
UNIT (Liu & Tuzel, 2017)	[NO] No	[YES] Yes	[NO] No	VAE-GAN
MUNIT (Huang et al., 2018)	[NO] No	[YES] Yes	[NO] No	Multimodal VAE
StarGAN (Choi et al., 2018)	[NO] No	[YES] Yes	[YES] Yes	Single-G multi-D
StarGAN v2 (Choi et al., 2020)	[NO] No	[YES] Yes	[YES] Yes	Mapping network
Our Implementation	[NO] No	[YES] Yes	[NO] No	ResNet + PatchGAN

7.3 Qualitative Comparison

Our implementation vs. original CycleGAN paper:

- **Horse <-> Zebra:** Our model trained for 300 epochs successfully applies zebra stripe patterns to horse bodies. The stripe color, distribution, and body conformance are visually consistent with the original paper results. Some background bleeding (texture bleeding from horse body to background) occurs, which is also observed in the original paper.
- **Monet <-> Photo:** Translating photos to Monet-style correctly introduces painterly brushstroke textures and characteristic blue-green color palettes. Photo reconstruction from Monet paintings produces realistic-looking landscapes. Results after 400 epochs show good stylistic consistency.

- **Summer <-> Winter:** Snow appearance and seasonal color shift (green->grey/white) are applied effectively. Trained for 250 epochs; results show seasonal texture adaptation with preserved scene structure.

Advantages of CycleGAN over Pix2Pix:

- No need for costly paired dataset collection
- Can learn from naturally occurring image collections (Monet paintings scraped from the web, wildlife photos)
- Bidirectional translation from a single training run

Limitations observed:

- Translation quality is bounded by the 256x256 training resolution
- Geometric transformations (e.g., changing dog breed shape) fail, only texture/style changes work reliably
- Mode collapse occasionally seen for the summer->winter direction in early epochs
- The model does not generalize to domain-specific object categories (e.g., cannot add or remove objects)

7.4 Subsequent SOTA Progress

Since CycleGAN (2017), the state of the art has advanced significantly:

- **MUNIT (2018):** Disentangles content and style codes, enabling multimodal outputs (multiple valid translations per input).
- **StarGAN v2 (2020):** Single model handles multiple domains simultaneously with style-code injection.
- **SPADE / GauGAN (2019):** Spatially adaptive normalization for high-resolution, semantically controlled synthesis.
- **Diffusion-based methods (2022-2025):** Methods like CycleDiffusion and InstructPix2Pix surpass GAN-based methods in FID and perceptual quality, but require more compute and the stochastic inference process is harder to control deterministically.

Our CycleGAN implementation remains a strong, interpretable, and computationally accessible baseline for unpaired domain translation tasks.

References

1. Zhu, J.-Y., Park, T., Isola, P., & Efros, A. A. (2017). *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks.* ICCV 2017. [arXiv:1703.10593]
2. Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2017). *Image-to-Image Translation with Conditional Adversarial Networks (Pix2Pix).* CVPR 2017.
3. Goodfellow, I., et al. (2014). *Generative Adversarial Nets.* NeurIPS 2014.
4. Liu, M.-Y., & Tuzel, O. (2016). *Coupled Generative Adversarial Networks (CoGAN).* NeurIPS 2016.
5. Liu, M.-Y., Breuel, T., & Kautz, J. (2017). *Unsupervised Image-to-Image Translation Networks (UNIT).* NeurIPS 2017.
6. Huang, X., Liu, M.-Y., Belongie, S., & Kautz, J. (2018). *Multimodal Unsupervised Image-to-Image Translation (MUNIT).* ECCV 2018.
7. Choi, Y., et al. (2020). *StarGAN v2: Diverse Image Synthesis for Multiple Domains.* CVPR 2020.
8. Ulyanov, D., Vedaldi, A., & Lempitsky, V. (2016). *Instance Normalization: The Missing Ingredient for Fast Stylization.* arXiv preprint arXiv:1607.08022.
9. Shrivastava, A., et al. (2017). *Learning from Simulated and Unsupervised Images through Adversarial Training.* CVPR 2017. [Replay Buffer]

10. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition.* CVPR 2016. [ResNet]